

Green Node — MVP 01

Complete Raspberry Pi 4 + ESP32 Simulator Implementation Guide

Goal: Build the first working proof of the Green Node edge-computing concept: a Raspberry Pi 4 receives greenhouse telemetry, stores it locally, applies transparent rules, and sends actuator commands through MQTT.

Basis: This guide follows the uploaded Green Node presentation. It specifies Raspberry Pi 4/5 as the hub, ESP32/Pico W as sub-nodes, MQTT/Mosquitto for communication, local sensor-history storage, and rules before ML/edge AI.

1. MVP 01 Goal

MVP 01 proves that many greenhouse devices can communicate through one local Green Node hub. The hub receives sensor data, stores it, makes a simple rule-based decision, and controls a virtual actuator without requiring the cloud.

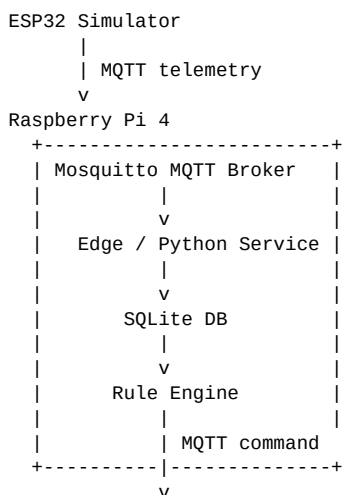
The presentation describes Green Node as a small edge-computing hub connecting, coordinating and automating existing greenhouse devices. It identifies device fragmentation and unreliable connectivity as core problems.

MVP 01 should not attempt full AI, every industrial protocol, a real greenhouse, or a complete production application. The presentation places core connectivity and rules before the pilot and ML/edge-AI stages.

2. Scope

Component	MVP 01
Hub	Raspberry Pi 4
Device layer	ESP32 simulator
Messaging	MQTT + Mosquitto
Storage	SQLite
Intelligence	Transparent rules
Actuators	Virtual pump + fan
Cloud	Not required
ML/AI	Not required yet
Real sensors	Not required yet

3. Architecture



Virtual Pump/Fan

Optional dashboard:

Browser -> Django REST API -> SQLite/MQTT

The complete demo loop is: sensor data → MQTT → Pi → local storage → rule decision → MQTT command → virtual actuator.

4. Hardware and Network

Raspberry Pi 4

- Raspberry Pi OS 64-bit
- Wi-Fi or Ethernet
- Keyboard + monitor for setup

i5 PC

- Development
- ESP32 simulator
- Browser/dashboard
- Git

Network:

PC <---- local LAN/Wi-Fi ----> Raspberry Pi 4

Use the i5 as the development machine and the Raspberry Pi as the actual edge hub. This makes the edge-processing demonstration meaningful.

5. Prepare Raspberry Pi

After installing Raspberry Pi OS and connecting to the local network, update the system:

```
sudo apt update
sudo apt full-upgrade -y
sudo reboot
```

After reboot:

```
hostname
hostname -I
python3 --version
git --version
```

Record the Pi's local IP address; the simulator will use it as the MQTT broker address.

6. Install Mosquitto

```
sudo apt update
sudo apt install -y mosquitto mosquitto-clients
sudo systemctl enable mosquitto
sudo systemctl start mosquitto
sudo systemctl status mosquitto
```

Test in two Pi terminals:

```
# Terminal 1
mosquitto_sub -h localhost -t greennode/test

# Terminal 2
mosquitto_pub -h localhost -t greennode/test -m "hello-green-node"
```

If the subscriber prints the message, MQTT is working.

7. MQTT Topic Design

```
greennode/{farm_id}/{node_id}/telemetry
greennode/{farm_id}/{node_id}/status
greennode/{farm_id}/{node_id}/command
greennode/{farm_id}/{node_id}/event
```

Example:

```
greennode/farm01/node01/telemetry
greennode/farm01/node01/command
```

Telemetry:

```
{  
  "device_id": "node01",  
  "temperature": 29.2,  
  "humidity": 70.0,  
  "soil_moisture": 25.0,  
  "light": 720  
}
```

Command:

```
{  
  "device_id": "node01",  
  "actuator": "pump",  
  "state": "ON",  
  "reason": "soil_moisture_below_threshold"  
}
```

8. Create Project

```
mkdir -p ~/greennode-mvp01
cd ~/greennode-mvp01
python3 -m venv .venv
source .venv/bin/activate
pip install paho-mqtt
pip freeze > requirements.txt
```

Recommended structure:

```
greennode-mvp01/
├── .venv/
├── config.py
├── mqtt_client.py
├── database.py
├── rules.py
├── edge_service.py
├── simulator.py
├── actuator_simulator.py
├── requirements.txt
├── data/
├── └── greennode.db
└── logs/
```

9. SQLite Local Storage

Use SQLite for MVP 01 so sensor history is stored on the edge hub. Suggested tables:

```
devices
  id, device_id, name, device_type, status, last_seen

sensor_readings
  id, device_id, temperature, humidity,
  soil_moisture, light, timestamp

actuator_events
  id, device_id, actuator, state,
  reason, timestamp
```

Database initialization:

```
# database.py
import sqlite3
from pathlib import Path

DB = Path("data/greennode.db")
DB.parent.mkdir(exist_ok=True)

def connect():
    return sqlite3.connect(DB)

def init_db():
    con = connect()
    cur = con.cursor()
    cur.execute("""
CREATE TABLE IF NOT EXISTS sensor_readings (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  device_id TEXT NOT NULL,
  temperature REAL,
  humidity REAL,
  soil_moisture REAL,
  light REAL,
  timestamp TEXT NOT NULL
)
""")
    cur.execute("""
CREATE TABLE IF NOT EXISTS actuator_events (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  device_id TEXT NOT NULL,
  actuator TEXT NOT NULL,
  state TEXT NOT NULL,
  reason TEXT,
  timestamp TEXT NOT NULL
)
""")
    con.commit()
```

```
con.close()
```

10. Rule Engine

The presentation's first intelligence stage is a simple transparent rule, such as starting irrigation when soil moisture falls below a threshold. Start with rules that are easy to explain and test.

```
SOIL_LOW = 30.0
SOIL_STOP = 40.0
TEMP_HIGH = 32.0

if soil_moisture < SOIL_LOW:
    pump = "ON"
elif soil_moisture >= SOIL_STOP:
    pump = "OFF"

fan = "ON" if temperature > TEMP_HIGH else "OFF"
```

The different pump thresholds provide a simple hysteresis band, reducing rapid ON/OFF switching around one value.

11. ESP32 Simulator

The simulator stands in for an ESP32 sub-node until physical hardware is available. It sends changing sensor values to the Pi's MQTT broker.

```
# simulator.py
import json, random, time
import paho.mqtt.client as mqtt

BROKER = "RASPBERRY_PI_IP"
TOPIC = "greennode/farm01/node01/telemetry"

client = mqtt.Client()
client.connect(BROKER, 1883, 60)

soil = 35.0

while True:
    soil += random.uniform(-2.5, 2.0)
    soil = max(15, min(60, soil))

    payload = {
        "device_id": "node01",
        "temperature": round(random.uniform(25,35),2),
        "humidity": round(random.uniform(55,85),2),
        "soil_moisture": round(soil,2),
        "light": round(random.uniform(300,900),2)
    }

    client.publish(TOPIC, json.dumps(payload))
    print(payload)
    time.sleep(5)
```

Replace RASPBERRY_PI_IP with the Pi's actual local IP address.

12. Edge Service

The edge service subscribes to telemetry, saves it, evaluates rules, and publishes actuator commands.

```
# edge_service.py
import json
from datetime import datetime, timezone
import paho.mqtt.client as mqtt
from database import init_db, connect

BROKER = "localhost"
TELEMETRY = "greennode/+/+/telemetry"
COMMAND = "greennode/farm01/node01/command"

SOIL_LOW = 30.0
SOIL_STOP = 40.0
TEMP_HIGH = 32.0

def publish_actuator(client, device_id, actuator, state, reason):
```

```

payload = {
    "device_id": device_id,
    "actuator": actuator,
    "state": state,
    "reason": reason
}
client.publish(COMMAND, json.dumps(payload))

def on_message(client, userdata, msg):
    data = json.loads(msg.payload.decode())
    now = datetime.now(timezone.utc).isoformat()

    con = connect()
    con.execute(
        """INSERT INTO sensor_readings
        (device_id, temperature, humidity, soil_moisture, light, timestamp)
        VALUES (?, ?, ?, ?, ?, ?)""",
        (data["device_id"], data["temperature"], data["humidity"],
        data["soil_moisture"], data["light"], now)
    )
    con.commit()
    con.close()

    if data["soil_moisture"] < SOIL_LOW:
        publish_actuator(client, data["device_id"], "pump", "ON",
            "soil_moisture_below_threshold")
    elif data["soil_moisture"] >= SOIL_STOP:
        publish_actuator(client, data["device_id"], "pump", "OFF",
            "soil_moisture_recovered")

    fan = "ON" if data["temperature"] > TEMP_HIGH else "OFF"
    publish_actuator(client, data["device_id"], "fan", fan,
        "temperature_rule")

init_db()
client = mqtt.Client()
client.on_message = on_message
client.connect(BROKER, 1883, 60)
client.subscribe(TELEMETRY)
client.loop_forever()

```

13. Virtual Actuator

```

# actuator_simulator.py
import json
import paho.mqtt.client as mqtt

BROKER = "RASPBERRY_PI_IP"
TOPIC = "greennode/farm01/node01/command"

def on_message(client, userdata, msg):
    data = json.loads(msg.payload.decode())
    print(
        f"ACTUATOR: {data['actuator']} -> {data['state']} | "
        f"reason={data.get('reason', '')}"
    )

client = mqtt.Client()
client.on_message = on_message
client.connect(BROKER, 1883, 60)
client.subscribe(TOPIC)

print("Virtual actuator online...")
client.loop_forever()

```

Replace RASPBERRY_PI_IP with the Pi IP address.

14. Run the Complete MVP

```

# Raspberry Pi
sudo systemctl start mosquitto
cd ~/greennode-mvp01
source .venv/bin/activate
python edge_service.py

# Development PC – terminal 1
python actuator_simulator.py

```

```
# Development PC – terminal 2
python simulator.py
```

Expected loop: simulator publishes telemetry → Pi receives it → SQLite stores it → rule engine decides → Pi publishes command → actuator simulator prints the new state.

15. Demonstration Scenarios

A. Normal
soil = 45%, temperature = 28°C
Expected: pump OFF, fan OFF

B. Dry soil
soil = 25%, temperature = 28°C
Expected: pump ON

C. Hot greenhouse
soil = 45%, temperature = 34°C
Expected: pump OFF, fan ON

D. Recovered soil
soil = 42%
Expected: pump OFF

16. Optional Django Dashboard

After the core loop works, add Django REST Framework for a web/mobile interface. Useful first endpoints:

```
GET /api/devices/
GET /api/readings/
GET /api/actuators/
GET /api/status/
POST /api/actuators/{device_id}/command/
```

A dashboard can display current sensor values, recent readings, device status and actuator state. Manual control can publish an MQTT command.

17. Security

For the prototype, keep MQTT on the trusted local network. Before exposing it outside the LAN, configure authentication, authorization and per-device topic permissions. Keep human dashboard authentication separate from device MQTT credentials.

18. Offline Test

Offline operation is a key part of the Green Node concept. Once the local loop works, disconnect the Pi from the internet while keeping the local network available. The simulator should still be able to send data and the Pi should still process rules locally.

```
Internet ON:
telemetry -> rule -> actuator
```

```
Internet OFF:
telemetry -> rule -> actuator
```

```
Reconnect:
local history remains available
```

Do not claim full offline capability until this test has actually been performed.

19. Automatic Startup

After manual testing works, run the edge service automatically with systemd.

```
sudo nano /etc/systemd/system/greennode-edge.service

[Unit]
Description=Green Node Edge Service
After=network-online.target mosquitto.service
Wants=network-online.target

[Service]
User=pi
WorkingDirectory=/home/pi/greennode-mvp01
ExecStart=/home/pi/greennode-mvp01/.venv/bin/python edge_service.py
Restart=always
RestartSec=3

[Install]
WantedBy=multi-user.target

sudo systemctl daemon-reload
sudo systemctl enable greennode-edge
sudo systemctl start greennode-edge
sudo systemctl status greennode-edge
journalctl -u greennode-edge -f
```

Change the username/path if your Raspberry Pi account is different.

20. Testing Checklist

Test	Expected
Mosquitto	Broker active
MQTT	Telemetry reaches Pi
Storage	Readings saved locally
Dry-soil rule	Pump ON command
Recovery rule	Pump OFF command
Temperature rule	Fan ON command
Actuator	Command received
Internet off	Local loop continues
Pi restart	Service/database recover
Multiple nodes	Topics stay separated

21. Common Problems

Simulator cannot connect: verify Pi IP, local network, Mosquitto status, and port 1883.

No MQTT message: check exact topic spelling; MQTT topics are case-sensitive.

Python import error: activate the virtual environment and run `pip install -r requirements.txt`.

Database not updating: check valid JSON, field names and write permissions.

Actuator does not react: verify command topic and JSON field names.

22. What Counts as Complete?

1. Raspberry Pi boots.
2. Mosquitto runs.
3. Edge service runs.
4. ESP32 simulator sends telemetry.
5. Pi receives telemetry.
6. Reading is saved locally.

7. Rule engine evaluates it.
8. Pi publishes a command.
9. Virtual actuator receives it.
10. Local loop works without internet.

When all ten steps work, MVP 01 has demonstrated the basic Green Node edge loop.

23. Evidence for Your Presentation

Capture:

- Raspberry Pi running Mosquitto
- MQTT telemetry messages
- Rule-engine decision in terminal
- SQLite sensor records
- Virtual pump/fan state changes
- Offline test
- Architecture diagram
- MQTT topic table
- Rule table
- Response-time measurements
- Short end-to-end demo video

These artifacts show a working, measurable system rather than only a concept.

24. Roadmap After MVP 01

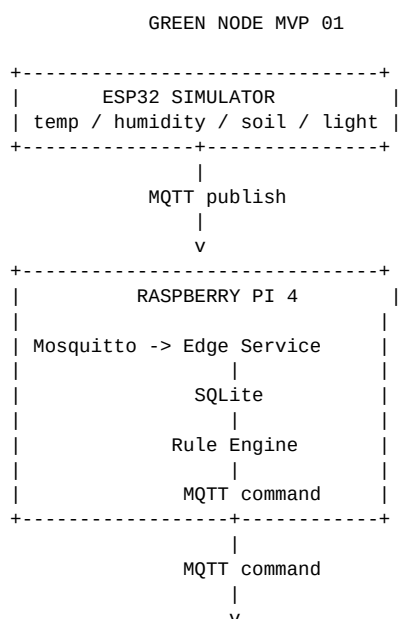
```

MVP 01
Core MQTT + local storage + rules
  |
  v
MVP 02
Unified farmer dashboard + device management
  |
  v
Pilot
Real ESP32 + sensors + actuators
  |
  v
ML / Edge AI
Compact local models + greenhouse-specific learning
  |
  v
Protocol adapters
Add device protocols one at a time

```

This sequence follows the presentation's six-month direction: core hub/connectivity, dashboard/rules, real greenhouse pilot, then ML/edge AI.

25. Final Architecture



```
+-----+
|          VIRTUAL ACTUATORS          |
|          Pump / Fan                  |
+-----+
```

LOCAL CONTROL LOOP:
Internet is not required.

The ESP32 simulator is only a stand-in for future ESP32 sub-nodes. The important component is the Raspberry Pi 4 acting as the Green Node edge hub.

Appendix — Minimum Commands

```
# Update
sudo apt update
sudo apt full-upgrade -y

# MQTT
sudo apt install -y mosquitto mosquitto-clients
sudo systemctl enable --now mosquitto

# Python environment
mkdir -p ~/greennode-mvp01
cd ~/greennode-mvp01
python3 -m venv .venv
source .venv/bin/activate
pip install paho-mqtt

# MQTT test
mosquitto_sub -h localhost -t greennode/test
mosquitto_pub -h localhost -t greennode/test -m "hello"

# Run edge service
python edge_service.py
```